

PROJECT SHOWCASE

ResearchLens

An AI SaaS-based Research Paper Analyzer built on a Hybrid Retrieval-Augmented Generation pipeline

LangChain

FAISS

Pinecone

BM25

FastAPI

Groq · Llama 3.3

PyPDF2

HuggingFace Transformers

Node.js

React

Vite

Tailwind

6

AI-POWERED
MODULES

Hybrid

FAISS + BM25
RETRIEVAL

3

INPUT
FORMATS

100%

CITED, SOURCE-GROUNDED
ANSWERS

What ResearchLens Does

The problem, the solution, and why it matters

ResearchLens is an end-to-end AI SaaS product that turns any academic paper — a PDF upload, a raw ArXiv/web URL, or even a scanned image — into an interactive, queryable knowledge base in under a minute. Instead of a researcher spending hours reading a 10-15 page paper to extract the problem statement, methodology, and results, ResearchLens ingests the document, builds a searchable vector index over it, and generates structured summaries, visual methodology flowcharts, cited Q&A, and study material — all grounded in the paper's actual text via Retrieval-Augmented Generation (RAG).

The core engineering problem this project solves: Large Language Models hallucinate when asked to explain long, technical documents from memory alone. ResearchLens solves this by never asking the LLM to "remember" the paper — instead, every answer is generated only from text chunks retrieved live from the paper itself, with every claim traceable back to a specific source chunk.

Who it's for

Students prepping for exams or paper discussions, researchers doing literature reviews, and engineers who need to quickly understand a paper's architecture before implementing it.

The six core features

Module	What it does
Quick Summary	One-click structured breakdown: Problem → Method → Results → Limitations → Future Work, plus auto-generated domain tags.
Quick Insights	Extracts the top 5 takeaways and a single one-line "elevator pitch" summary of the paper.
Methodology Flowchart	Auto-generates a visual Mermaid.js diagram of the paper's pipeline/architecture directly from retrieved chunks.
Chat Q&A	Free-form question answering with inline chunk citations and live-computed evaluation metrics (Faithfulness, Relevance, Context Precision) for every answer.
Full Paper Deep-Dive	Generates an expert-level, masterclass-style explanation of the entire paper end-to-end — background, methodology, experiments, results, and a glossary of key terms.
Study Prep / Quiz	Generates 5 technical interview-style questions with ideal, cited answers to test understanding of the paper.

41

CHUNKS INDEXED (SAMPLE RUN)

11

PAGES PROCESSED

32,622

CHARACTERS EMBEDDED

How It's Built

Full-stack RAG pipeline: ingestion → retrieval → generation → UI

End-to-end pipeline

- 1

Ingestion — PDF upload (parsed with PyPDF2), URL scraping (ArXiv or general web pages), or image OCR is normalized into raw text.
- 2

Chunking — Text is split into overlapping chunks sized to balance retrieval precision against context completeness.
- 3

Embedding — Each chunk is embedded using HuggingFace Transformer models into dense vector representations.
- 4

Hybrid Indexing — Vectors are indexed in FAISS for local, low-latency dense similarity search, with Pinecone available as a swap-in for scalable, cloud-hosted, multi-thousand-document indexing.
- 5

Hybrid Retrieval — A custom LangChain retriever combines FAISS (dense/semantic) with BM25 (sparse/keyword) search, so the system catches both conceptual matches and exact-term matches (e.g. "BLEU score", "self-attention").
- 6

Reranking — Retrieved candidates are reranked (visible in the UI as the "RERANKED" badge) to surface the most relevant chunks to the top before they're passed to the LLM.
- 7

Generation — Groq-hosted Llama 3.3 generates the final answer, summary, or flowchart, conditioned only on the retrieved, reranked chunks — with citations attached to every claim.
- 8

Serving — A FastAPI backend exposes all of this over REST endpoints, consumed by a React + Vite + Tailwind frontend.

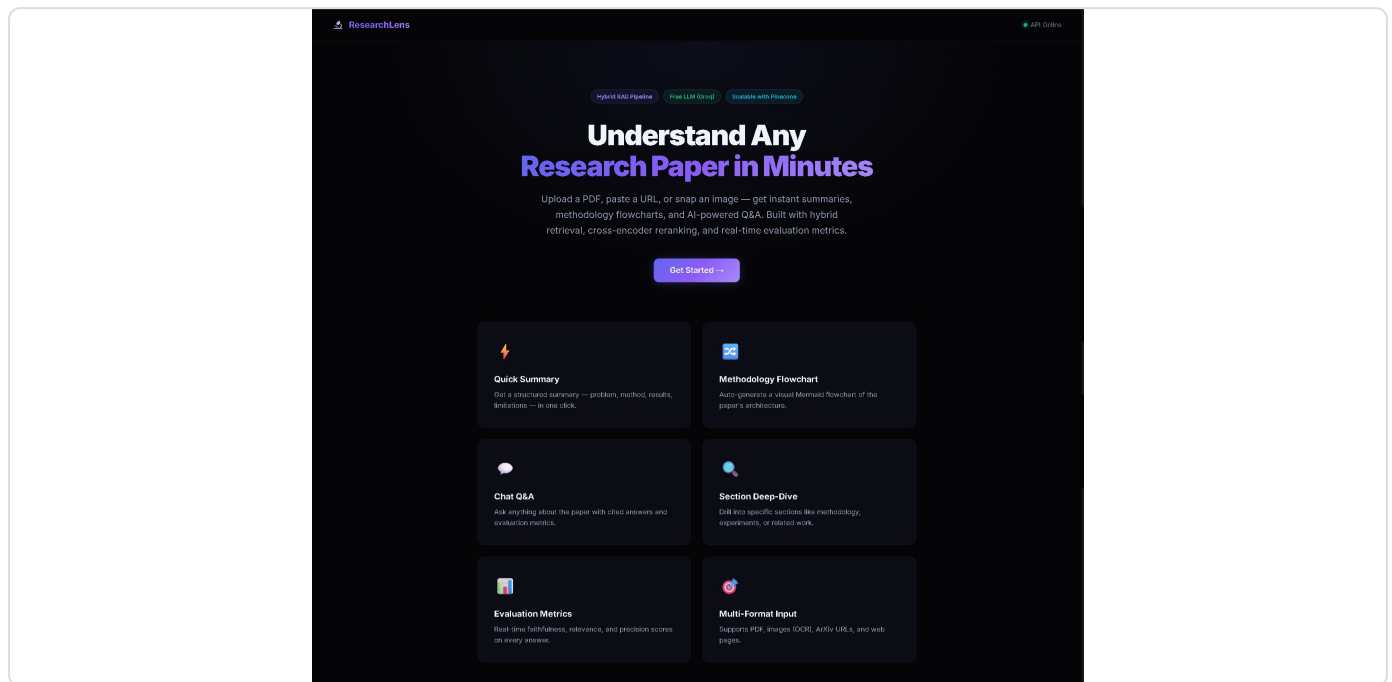
Why this architecture

Decision	Reasoning
Hybrid FAISS + BM25 retriever	Pure dense retrieval misses exact technical terms and numbers papers rely on (metric names, model names); pure keyword search misses paraphrased concepts. Combining both improves recall on technical text.
Groq (Llama 3.3) for inference	Groq's LPU inference is dramatically faster than typical GPU-hosted inference, which matters directly for a tool whose entire value proposition is speed — "understand any paper in minutes."
FAISS by default, Pinecone optional	FAISS keeps the tool free and fast for a single-session/local use case; Pinecone is offered as a drop-in for teams that need persistent, multi-thousand-document, cloud-scale indexing.
FastAPI backend	Async-first, automatic OpenAPI schema generation, and low overhead — a good fit for an I/O-bound pipeline that's constantly calling external embedding and LLM APIs.
Manual endpoint validation via Postman	Every API endpoint was manually validated for reliability before deployment, rather than relying purely on unit tests, to catch integration-level failure modes (timeouts, malformed responses).

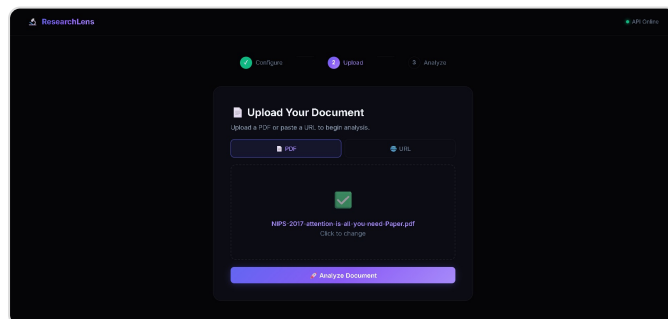
The abstraction-based approach: The retrieval and embedding layer is built with modular, swappable document loaders and embedding managers, so the same core pipeline can be pointed at PDFs, URLs, or images without rewriting the retrieval logic — a pattern directly reusable for large-scale payment or document-processing systems that need to abstract over heterogeneous input sources.

From Upload to Insight

A guided tour of the live product experience



Landing page — value proposition, capability badges, and the six core modules at a glance.



Step 2 — Upload a PDF (or paste a URL) to begin analysis.

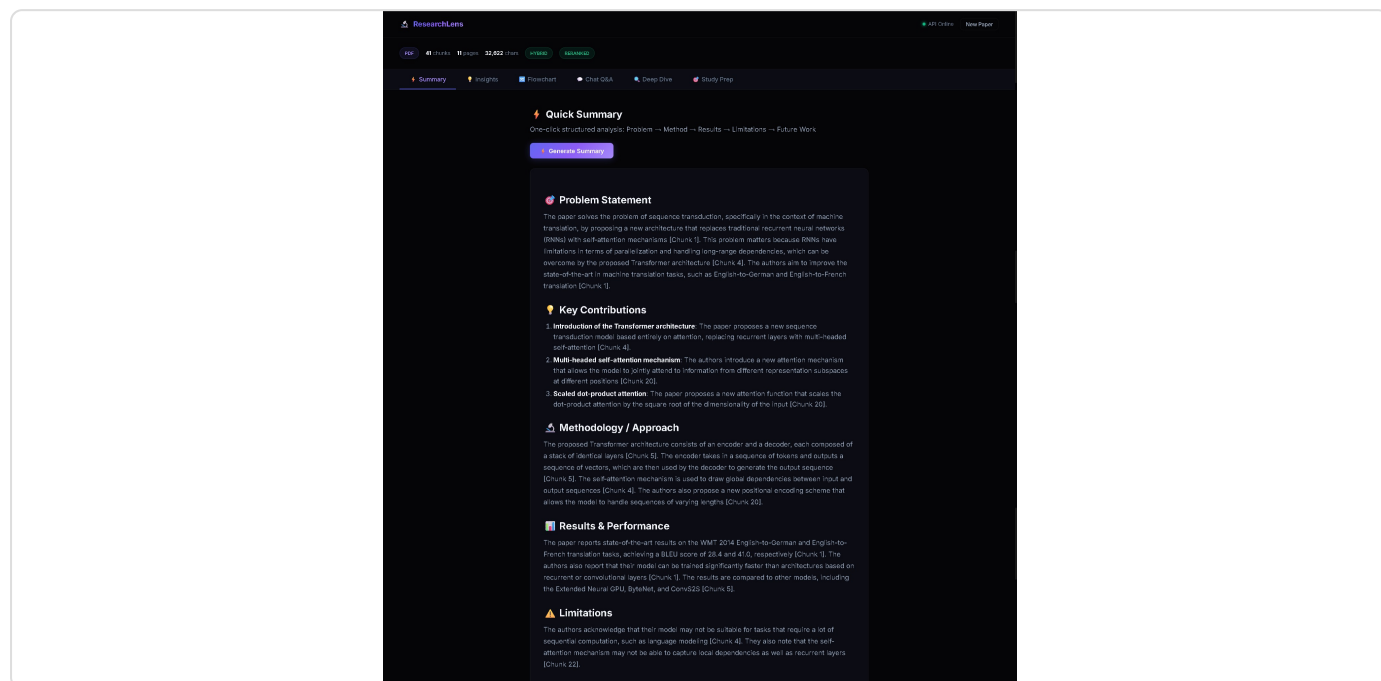


Multi-format input — the same pipeline accepts a direct paper URL.

The onboarding flow is a simple three-step wizard: **Configure** → **Upload** → **Analyze**. On configuration, the app already has a Groq API key set server-side, with an optional Pinecone key for users who want cloud vector storage instead of local FAISS — otherwise it falls back to FAISS automatically. This design keeps the "get started in one click" experience intact for a demo/free-tier user while still supporting a scalable, bring-your-own-key production path.

Structured Summary Output

RAG-grounded, citation-backed document understanding



Quick Summary — Problem Statement, Key Contributions, Methodology, Results, Limitations, Future Work, and Domain Tags, each claim tied to a specific retrieved chunk.

Every section of the summary is generated strictly from the 10 highest-scoring retrieved chunks shown below the summary (visible with their retrieval scores) — not from the model's pretraining knowledge of the paper. This is the practical difference between a RAG system and "just asking ChatGPT to summarize a paper": the output is auditable, and a user can expand the source chunks to verify any claim against the original text.

Insights & Visual Methodology

Distilled takeaways and auto-generated architecture diagrams

Future Work

The authors suggest that future work could involve investigating the use of restricted self-attention to improve computational performance for tasks involving very long sequences [Chunk 22]. They also propose exploring the use of more sophisticated compatibility functions, such as those based on neural networks [Chunk 30].

Domain Tags

#TransformerArchitecture #NLP #MachineTranslation #AttentionMechanisms #DeepLearning

9 10 source chunks retrieved

Chunk 2 - Score: -4.592

transformation problems such as language modeling and machine translation [29, 2, 6]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [31, 21, 10] -@equal contribution. Later...

Chunk 1 - Score: -11.243

reality. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation...

Chunk 20 - Score: -11.083

because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training. 4 Why Self-Attention In this section we compare various aspects of self-attention layers to the recurrent and convolutional layers...

Chunk 5 - Score: -5.102

In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more paralleliza...

Chunk 4 - Score: -9.236

sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency...

Chunk 29 - Score: -4.600

were written at 10-second intervals. For the big models, we averaged the last 20 checkpoints. We used beam search with a beam size of 4 and length penalty + 0.6 [25]. These hyperparameters were chosen after experimentation on the development set. We...

Chunk 30 - Score: -17.751

To evaluate the importance of different components of the Transformer, we varied our base model in different ways, measuring the change in performance on English-to-German translation on the development set, measured@20. We used beam search as described...

Chunk 37 - Score: -11.272

Ray Kurzweil, Neural machine translation in linear time, arXiv preprint arXiv:1610.10099v2, 2017. [16] Yoon Kim, Carl Denton, Luning Huang, and Alexander M. Rush, Structured attention networks, In International Conference on Learning Representations...

Chunk 23 - Score: -7.809

executed operations, whereas a recurrent layer requires O(n) sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence length n is smaller than the representation dimension d...

Chunk 32 - Score: -10.344

In Table 3 rows (B), we observe that reducing the attention key size degrades model quality. This suggests that determining compatibility is not easy and that a more sophisticated compatibility function than dot product may be beneficial. We further ex...

Quick Insights — top 5 takeaways plus a one-line summary, generated from a separate, smaller retrieval pass (5 chunks) tuned for high-signal extraction.

ResearchLens

PDF 41 chunks 11 pages 32,622 chars HYBRID REARRANGED

Summary Insights Flowchart Chat Q&A Deep Dive Study Prep

Quick Insights

Get the top 5 key takeaways and concepts from the paper instantly.

Extract Insights

5 Key Insights from this Paper

- Transformer Model Architecture:** The Transformer model is the first sequence transduction model based entirely on attention, replacing recurrent layers with multi-headed self-attention, allowing for significantly faster training times for translation tasks [Chunk 32].
- Importance of Attention Key Size:** Reducing the attention key size hurts model quality, suggesting that determining compatibility is not easy and a more sophisticated compatibility function may be beneficial [Chunk 32].
- Multi-Head Attention:** Varying the number of attention heads and attention key and value dimensions shows that single-head attention is 0.9 BLEU worse than the best setting, and quality also drops off with too many heads [Chunk 30].
- Parallelization Limitations of Recurrent Models:** Recurrent models are inherently sequential, precluding parallelization within training examples, which becomes critical at longer sequence lengths due to memory constraints [Chunk 3].
- Effectiveness of Dropout and Positional Encoding:** Dropout is very helpful in avoiding overfitting, and replacing sinusoidal positional encoding with learned positional embeddings results in nearly identical performance to the base model [Chunk 32].

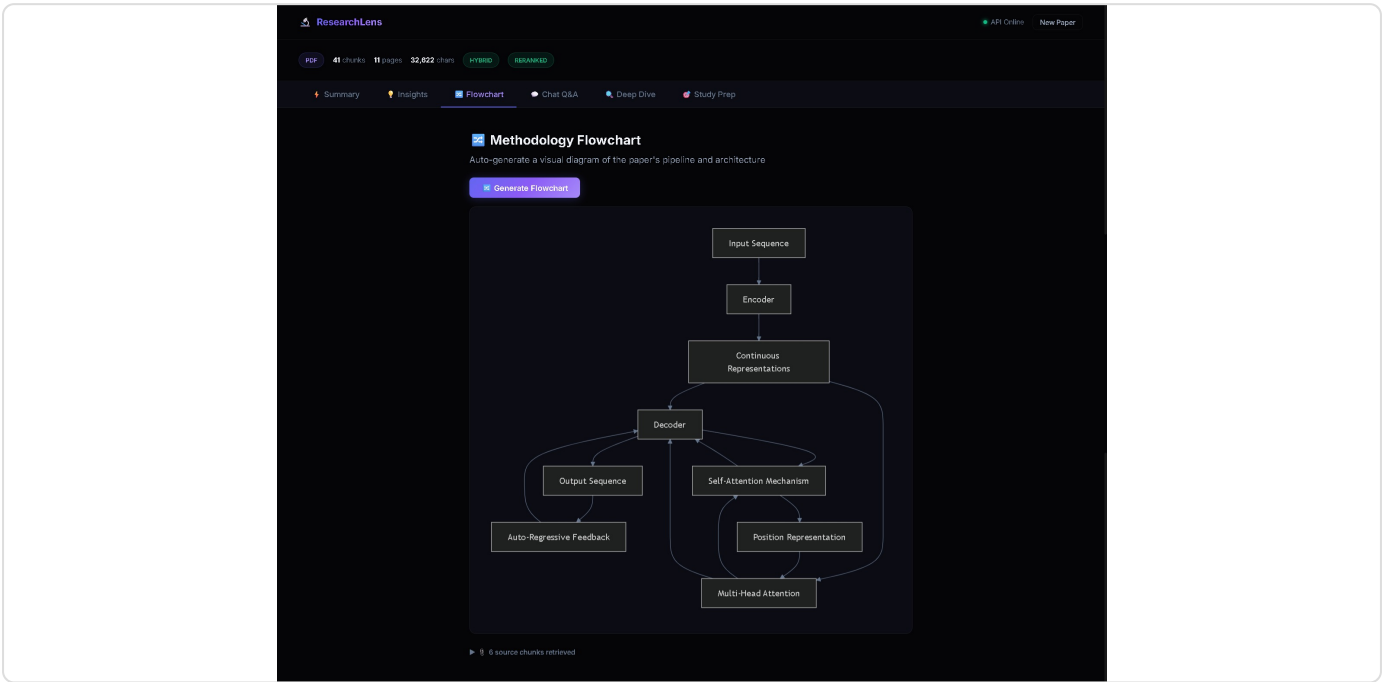
One-line takeaway: The Transformer model, based entirely on attention, achieves state-of-the-art results in translation tasks and can be trained significantly faster than recurrent or convolutional architectures [Chunk 32].

9 5 source chunks retrieved

Methodology Flowchart — a Mermaid.js diagram generated by asking the LLM to describe the paper's architecture as a directed graph, then rendering it visually. This turns a dense "Methodology" section into a diagram a reader can follow in seconds.

Chat Q&A with Live Evaluation Metrics

Ask anything — every answer is cited and scored in real time

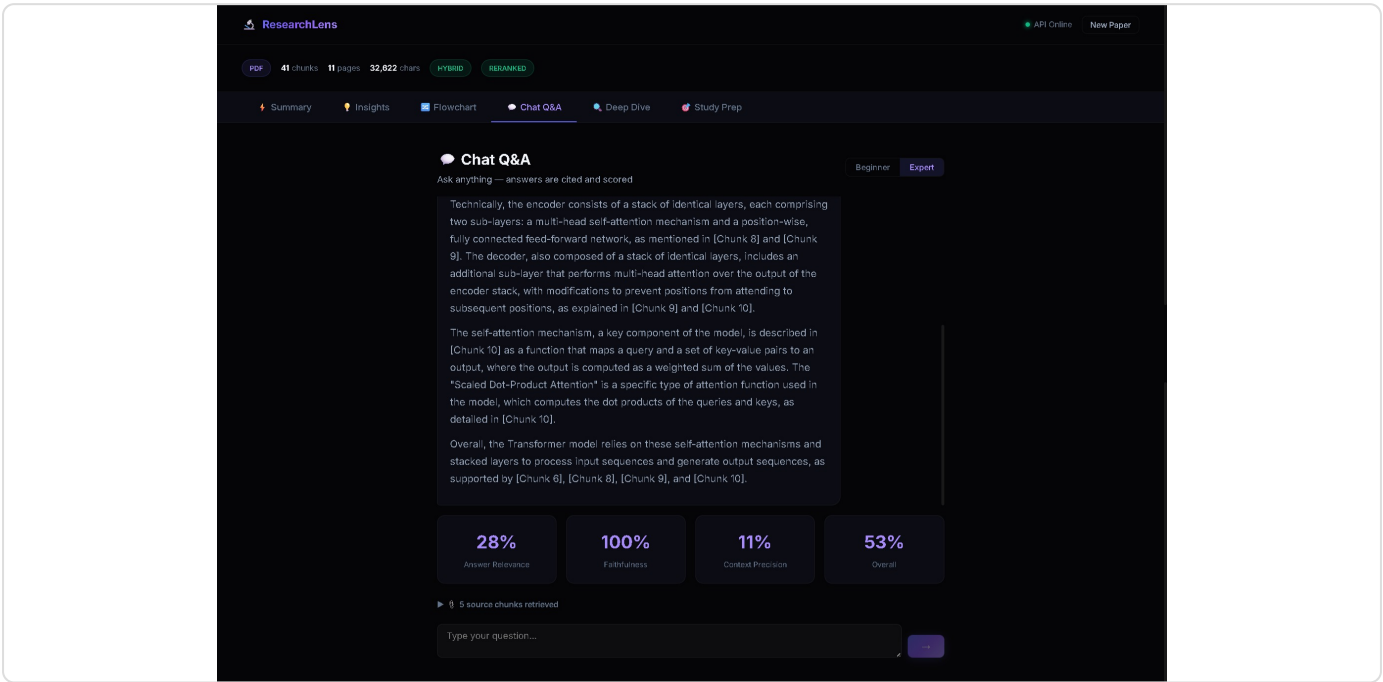


Chat Q&A (Expert mode) — free-form questions answered with inline chunk citations, alongside real-time Faithfulness, Answer Relevance, Context Precision, and an Overall score computed for that specific answer.

Why this matters technically: Most RAG demos stop at "retrieve and generate." ResearchLens goes a step further by computing RAG evaluation metrics (a faithfulness/groundedness check, a relevance check, and a context precision check) on every single answer, live, so the user — and the interviewer — can see exactly how well-grounded each response is, not just trust it blindly.

Deep-Dive Masterclass & Study Prep

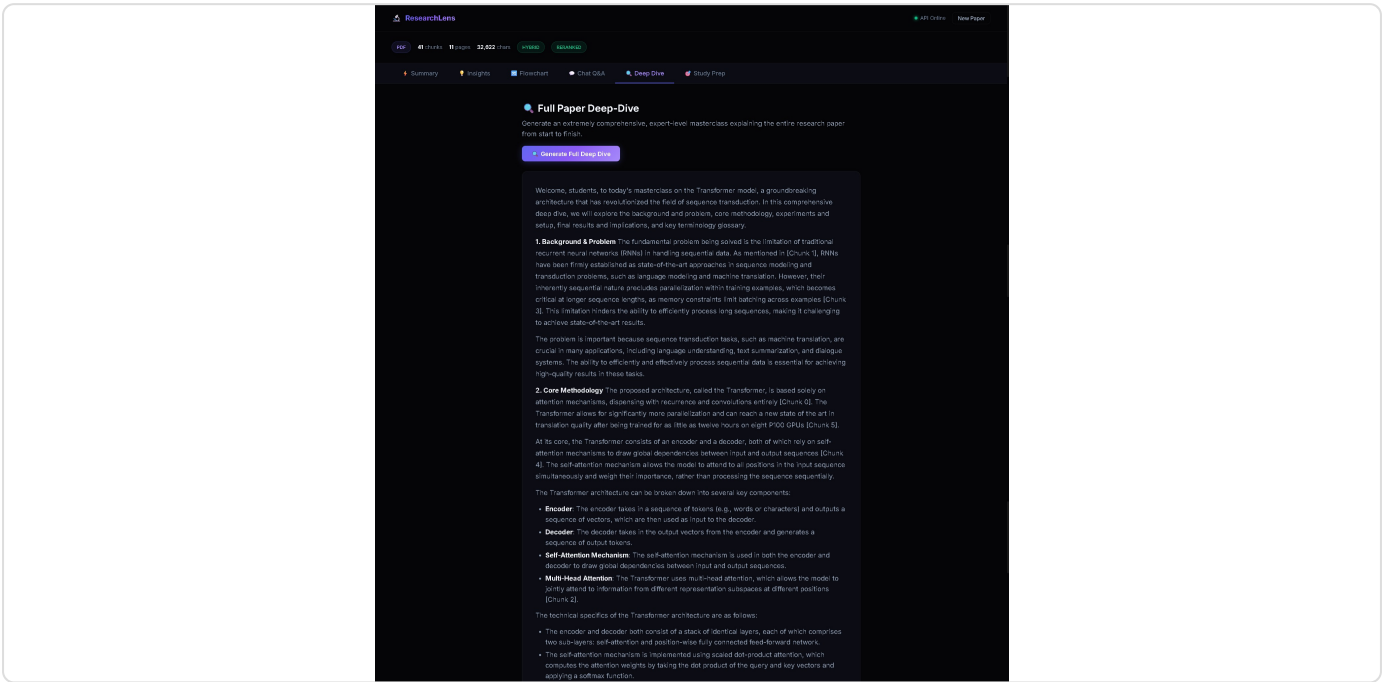
Turning a paper into a full learning experience



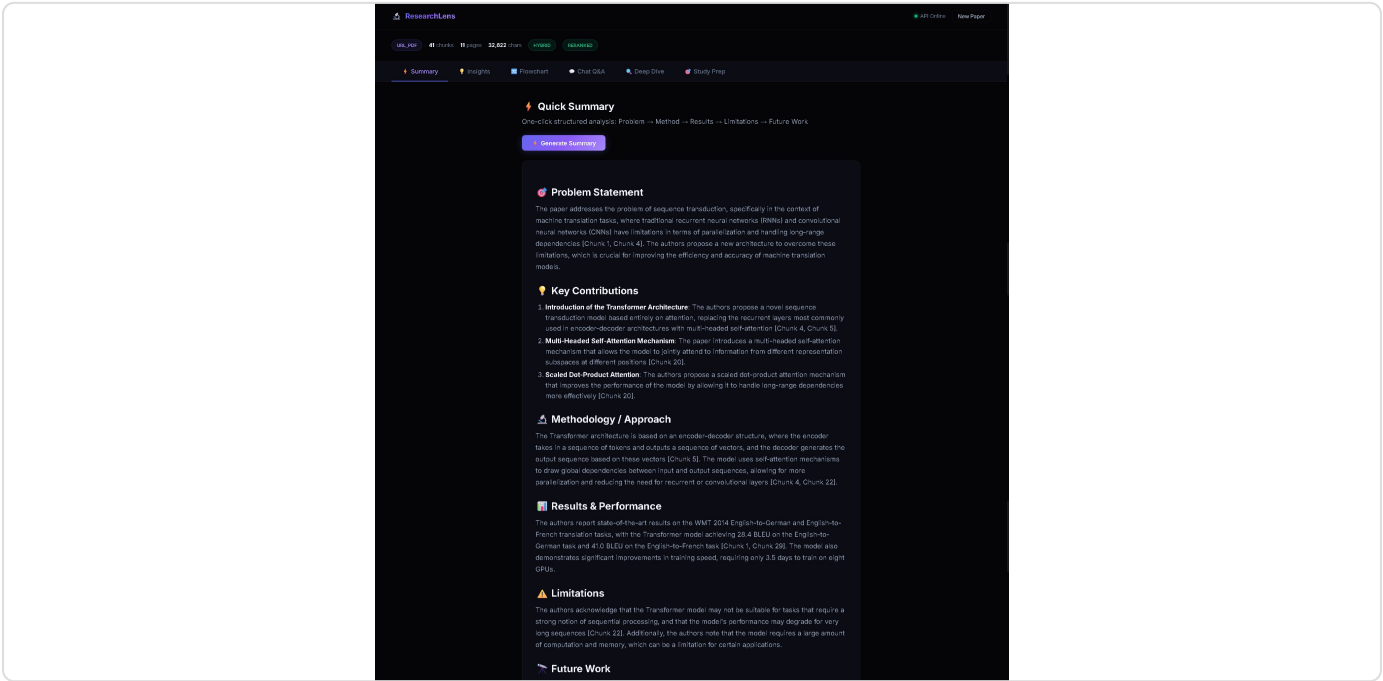
Full Paper Deep-Dive — an expert-level, section-by-section masterclass covering background, methodology, experimental setup, results, and a glossary, synthesized from 8 retrieved chunks spanning the whole paper.

Interview & Study Prep

Self-testing, generated directly from the source paper



Study Prep — generates exactly 5 technical questions with ideal, cited answers, letting a student or researcher test their own understanding before a meeting or exam.



The same Quick Summary pipeline run against a raw URL input instead of an uploaded PDF — proving the ingestion layer is genuinely format-agnostic, not just a PDF-only pipeline with a URL field bolted on.

Under the Hood

Tools, and the specific engineering decisions worth discussing

Technology stack



Three engineering highlights worth leading with

1. Hybrid retrieval, not just vector search

Combining FAISS (dense) with BM25 (sparse) inside a single LangChain retriever directly addresses the well-known weakness of pure embedding search on technical/scientific text — exact terminology and numbers get lost in dense-only retrieval. This is a production-grade retrieval pattern, not a toy RAG demo.

2. Answers are auditable, not just generated

Every summary, insight, flowchart, and chat answer carries chunk-level citations plus retrieval scores, and Chat Q&A additionally computes live faithfulness/relevance/precision metrics per answer — turning "trust the AI" into "verify the AI," which is the difference between a toy and a tool a researcher could actually rely on.

3. Format-agnostic ingestion with a swappable storage layer

The same retrieval pipeline accepts PDFs, URLs, and OCR'd images, and the vector store itself swaps between local FAISS and cloud Pinecone with no change to retrieval logic — a genuinely reusable abstraction, not a hardcoded single-source pipeline.

Production considerations addressed

- CORS security and robust error handling on the FastAPI backend for safe browser-based access.
- Manual validation of every API endpoint using Postman prior to deployment, to catch integration failures before they reached the frontend.
- Clean deployment on Node.js for the frontend, decoupled from the Python backend, so each layer can scale and deploy independently.

Note on live demo access: The hosted live link for ResearchLens may be temporarily unavailable at the time of this interview. This document — along with the accompanying screenshots — reflects the fully functional, deployed product and is intended to let the interview proceed productively without live access.